# Week 2 Task Submission: Database & API Development

Name: Rishav Dev Tiwari

Program: Fusemachines AI Fellowship 2026

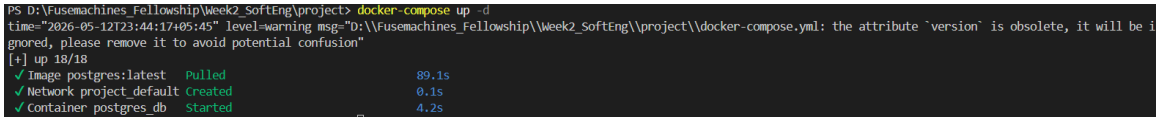
## 1. Project Overview

This project implements a fully functional REST API for the ClassicModels database utilizing a 4-layer clean architecture (Connection, Data Format, Logic, and API Layer). The application adheres to the Twelve-Factor App methodology, specifically focusing on Dependency Management, Config Management, Backing Services, and Concurrency.

## 2. Environment & Database Setup (Factor II, III, & IV)

The application utilizes Docker to ensure a consistent environment and to manage PostgreSQL as an isolated backing service. Database credentials and variables are isolated in a .env file.

The database is containerized using docker-compose.yml. Upon the initial startup, the container automatically executes seed.sql via the /docker-entrypoint-initdb.d/ volume bind, successfully building all 8 tables and seeding them with data.




## 3. API Testing & Documentation

The API leverages FastAPI automatic interactive documentation (Swagger UI). Each of the 8 tables features its own tagged section containing GET and COUNT endpoints.

# ClassicModels API 0.1.0 OAS 3.1

/openapi.json

Week 2 Assignment API

## ProductLines

GET `/productlines/count` Get Productlines Count

GET `/productlines/` Get Productlines

## Customers

GET `/customers/count` Get Customers Count

GET `/customers/` Get Customers

## Dashboard

GET `/overall_counts` Get Overall Counts

# Dashboard

GET `/overall_counts` Get Overall Counts

**Parameters** Cancel

No parameters

Execute | Clear

**Responses**

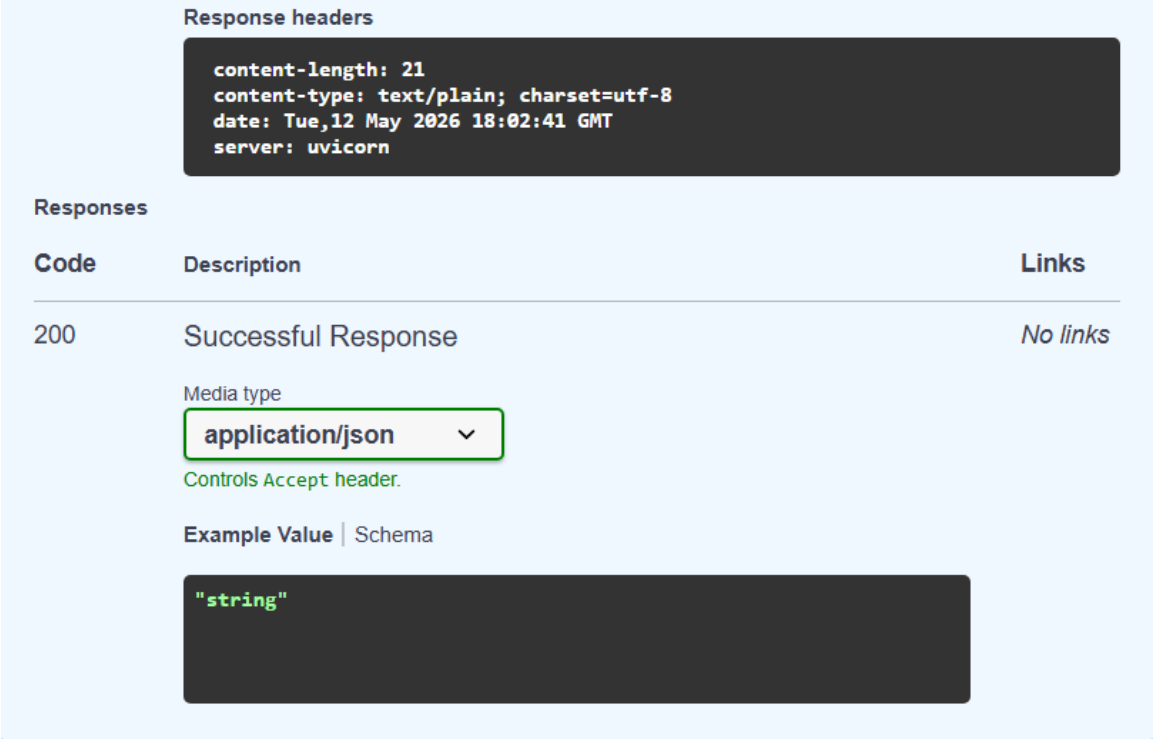
Curl

```
curl -X 'GET' \
  'http://localhost:8000/overall_counts' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/overall_counts
```

Server response




## 4. Concurrency Implementation (Factor VIII)

To achieve high performance under load, the application utilizes asynchronous concurrent processing for the master dashboard endpoint (/overall_counts).

Instead of querying the 8 database tables sequentially (which creates blocking delays), the API uses asyncio.gather() combined with asyncio.to_thread(). This allows the FastAPI server to dispatch all 8 SELECT COUNT(*) queries to the PostgreSQL database simultaneously, drastically reducing the total response time.

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [10368] using StatReload
INFO:     Started server process [34300]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:60425 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:60425 - "GET /openapi.json HTTP/1.1" 200 OK
2026-05-12 23:47:42,036 - api_logger - INFO - Incoming request: GET /overall_counts
2026-05-12 23:47:42,036 - api_logger - INFO - Starting concurrent tasks...
2026-05-12 23:47:42,045 - api_logger - INFO - Executing query: SELECT COUNT(*) FROM customers
```

Github Repository: https://github.com/rishavdevtiwari/fusemachines-fellowship.git